Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование рядов Тейлора»

Выполнил:
студент группы 3825Б1ПМии1
Семанин И.А.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Оглавление

Введение **3**
Постановка задачи **4**
Руководство пользователя **5**
Описание программной реализации **7**
Результаты экспериментов **10**
Заключение **13**
Литература **14**
Приложение **15**

Введение

В современной вычислительной математике разложение функций в степенные ряды, в частности в ряды Тейлора, является одним из фундаментальных инструментов вычислений. Данный метод позволяет заменять сложные элементарные функции (тригонометрические, экспоненциальные, логарифмические) на полиномы, что критически важно для реализации вычислений в программном обеспечении, где прямое вычисление этих функций может быть затруднено или ресурсозатратно.

Однако точность такого приближения напрямую зависит от двух факторов: количества членов ряда и порядка их суммирования. При сложении чисел различного порядка возникает вычислительная погрешность

В данной работе исследуется влияние порядка суммирования членов ряда Тейлора на точность вычислений. Сравниваются два подхода: суммирование слева (от первого члена к последнему) и суммирование справа (от последнего члена к первому). На основе полученных экспериментальных данных делается вывод о наиболее точном способе накопления суммы.

Постановка задачи

Разработать консольное приложение на языке программирования C, которое вычисляет приближенные значения функций с помощью их разложения в ряд Тейлора.

Программа должна обеспечивать:

1. Выбор пользователем функции для разложения:
   - $\sin(x)$
   - $\cos(x)$
   - e^x
   - $\ln(1+x)$
2. Ввод значения аргумента x и количества членов ряда n.
3. Вычисление членов ряда с использованием рекуррентных формул.
4. Вычисление двух типов частичных сумм:
   - **Сумма слева (sum_left):** накопление результатов последовательным сложением членов, начиная с первого.
   - **Сумма справа (sum_right):** накопление результатов последовательным сложением членов, начиная с последнего.
5. Вычисление абсолютной погрешности каждого приближения по сравнению с точным значением функции из стандартной библиотеки math.h.
6. Вывод результатов в виде таблиц для наглядного сравнения.
7. Анализ полученных данных и формулировка выводов о точности каждого метода.

Руководство пользователя

Запуск программы

После компиляции и запуска программы пользователю предлагается выбрать функцию для разложения:

```
chose:
1 - sin(x)
2 - cos(x)
3 - exp(x)
4 - ln(1+x)
```

Рис. 1 Меню приложения

Ввод данных

Пользователь вводит номер выбранной функции, затем значение аргумента x и количество членов ряда *n*:

```
ins x and n:
```

Рис. 2 Ввод данных

Вывод результатов

Программа выводит две таблицы:

1. **sum_left** — результаты суммирования слева
2. **sum_right** — результаты суммирования справа

Каждая строка таблицы содержит:

- Номер члена
- Значение накопленной суммы
- Абсолютную погрешность

Интерпретация результатов

- Для **левой суммы** таблицу следует читать **сверху вниз** — каждая следующая строка добавляет очередной член ряда.
- Для **правой суммы** таблицу следует читать **снизу вверх** — последняя строка соответствует первому члену, предпоследняя — сумме двух членов и т.д.

```
sin(x)

(sum_left)
0 2.000000 1.090703
1 0.666667 0.242631
2 0.933333 0.024036
3 0.907936 0.001361
4 0.909347 0.000050
5 0.909296 0.000001
6 0.909297 0.000000
7 0.909297 0.000000
8 0.909297 0.000000
9 0.909297 0.000000

(sum_right)
0 0.909297 0.000000
1 -1.090703 2.000000
2 0.242631 0.666667
3 -0.024036 0.933333
4 0.001361 0.907937
5 -0.000050 0.909347
6 0.000001 0.909296
7 -0.000000 0.909297
8 0.000000 0.909297
9 -0.000000 0.909297
```

Рис. 3 Пример выводимой таблицы для sin 2 радиан(n=100)

Описание программной реализации

Структура данных

Для организации работы с рядом определена структура x_n, где

int n - количество членов ряда

float* x - массив членов ряда

float* sum - массив частичных сумм

float* err - массив погрешностей

Функции заполнения членов ряда:

1) void fill_sin(x_n& main, float x_)
2) void fill_cos(x_n& main, float x_)
3) void fill_exp(x_n& main, float x_)
4) void fill_ln(x_n& main, float x_)

Функции принимает переменную типа данных x_n(& указывает на адрес переменной), а также вводимый пользователем x-аргумент sin/cos/ln/e^x (функции ничего не возвращают)

Для повышения эффективности вычислений и минимизации накопления ошибок округления используется **рекуррентный метод** вычисления членов ряда. Каждый следующий член вычисляется из предыдущего с помощью множителя, зависящего от *x* и номера итерации:

- **Для sin(x):** $a_{n+1} = -\frac{x^2}{(2n+2)(2n+3)} \cdot a_n$
- **Для cos(x):** $a_{n+1} = -\frac{x^2}{(2n+1)(2n+2)} \cdot a_n$
- **Для exp(x):** $a_{n+1} = \frac{x}{n+1} \cdot a_n$
- **Для ln(1+x):** $a_{n+1} = -a_n \cdot x \cdot \frac{n}{n+1}$

Рис. 4 Рекуррентные формулы для основных функций

Принцип работы:

1. Первый член (a_0) устанавливается фиксированным(1 или x)

2. Для каждого последующего члена вычисляется множитель через рекуррентную формулу

3. Новый член получается умножением предыдущего на этот множитель

Функции суммирования

void sum_left(x_n& main):

Функция принимает переменную типа данных x_n(& указывает на адрес переменной; функция ничего не возвращает)

Назначение: вычисление частичных сумм ряда путем последовательного сложения членов слева направо (от первого к последнему).

Принцип работы:

1. Первая частичная сумма (sum[0]) равна первому члену ряда
2. Для каждого последующего индекса k вычисляется сумма: sum[k] = x[k] + sum[k-1]
3. В результате sum[k] содержит сумму первых k+1 членов ряда

Особенности: этот метод начинает накопление с наибольшего по модулю члена (первого), поэтому погрешность монотонно убывает с каждым шагом.

void sum_right(x_n& main):

Функция принимает переменную типа данных x_n(& указывает на адрес переменной функция ничего не возвращает)

Назначение: вычисление «хвостовых сумм» ряда путем последовательного сложения членов справа налево (от последнего к первому).

Принцип работы:

1. Последняя частичная сумма (sum[n-1]) равна последнему члену ряда
2. Для каждого предыдущего индекса k вычисляется сумма: sum[k] = x[k] + sum[k+1]
3. В результате sum[k] содержит сумму членов с индекса k до конца (n-1)

Особенности:

- При чтении снизу вверх этот массив показывает накопление суммы в обратном порядке
- Последний элемент (sum[n-1]) соответствует последнему члену ряда
- Первый элемент (sum[0]) соответствует полной сумме всех членов

Функции вычисления погрешности

- void err_left(x_n& main, float vvod, float (*func)(float));

- void err_right(x_n& main, float vvod, float (*func)(float));

Функция принимает: переменную типа данных x_n(& указывает на адрес переменной), переменная vvod-аргумент sin/cos/ln/e^x, float (*func)(float)- это указатель на функцию(sin/cos/ln/e^x), вызывает переданную функцию, которая возвращает тип данных float

Функция ничего не возвращает

Назначение: вычисление абсолютной погрешности приближения для каждого шага суммирования.

Принцип работы:

1. Функция принимает указатель на структуру, значение аргумента и указатель на функцию вычисления точного значения

2. Для каждого индекса j вычисляется разность между полученной частичной суммой и точным значением

3. Вычисляется модуль разности с помощью функции abs() из библиотеки math.h

4. Результат сохраняется в массив err

Функции-обертки для математических функций

1) float sin_float(float x);
2) float cos_float(float x);
3) float exp_float(float x);
4) float ln_float(float x)

   Функция принимает вводимый пользователем x(возвращает тип данных float)

   Назначение: преобразование типов для функций стандартной библиотеки math.h.

Принцип работы: Стандартные функции sin, cos, exp, log работают с типом double. Эти функции-обертки преобразуют результат к типу float. Это обеспечивает корректное сравнение приближенных значений с точными.

Результаты экспериментов

Эксперименты проводились для различных функций и значений аргумента. Для наглядности представлены результаты для sin(2) и cos(2) при *n*=6, exp(4) n=13 и ln(1,5) n=8. В результате были составлены графики

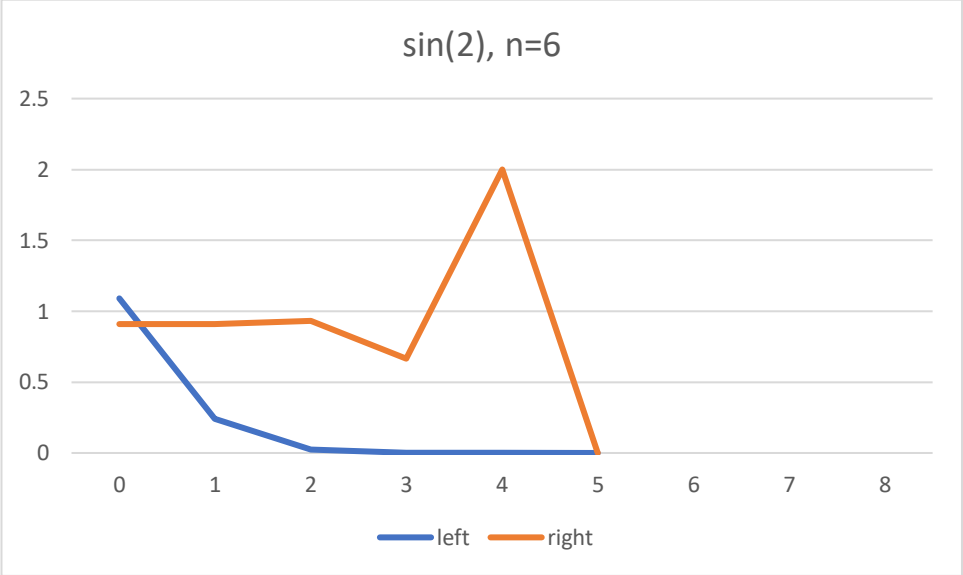

Рис. 5 погрешности синуса

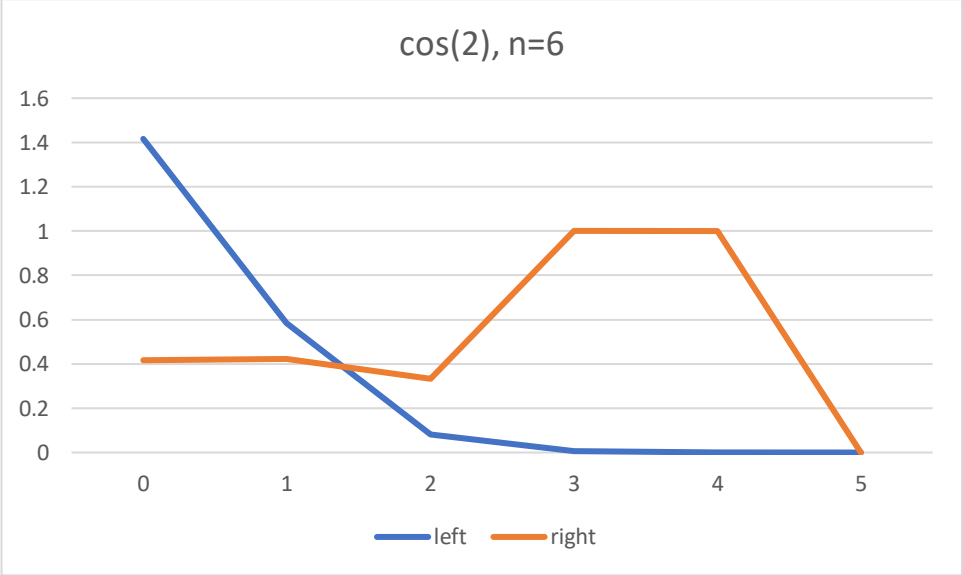

Рис. 6 погрешности косинуса

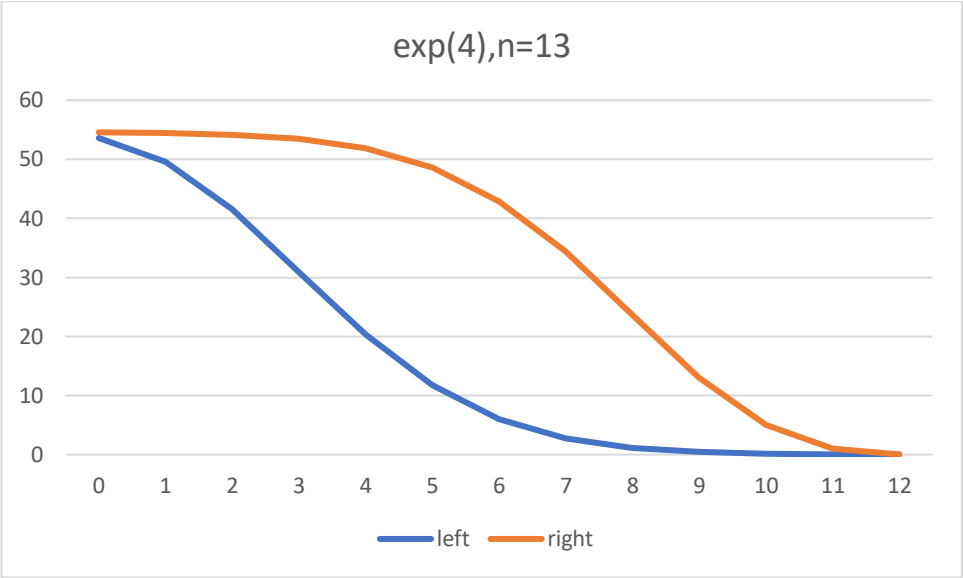

Рис. 7 погрешности экспоненты

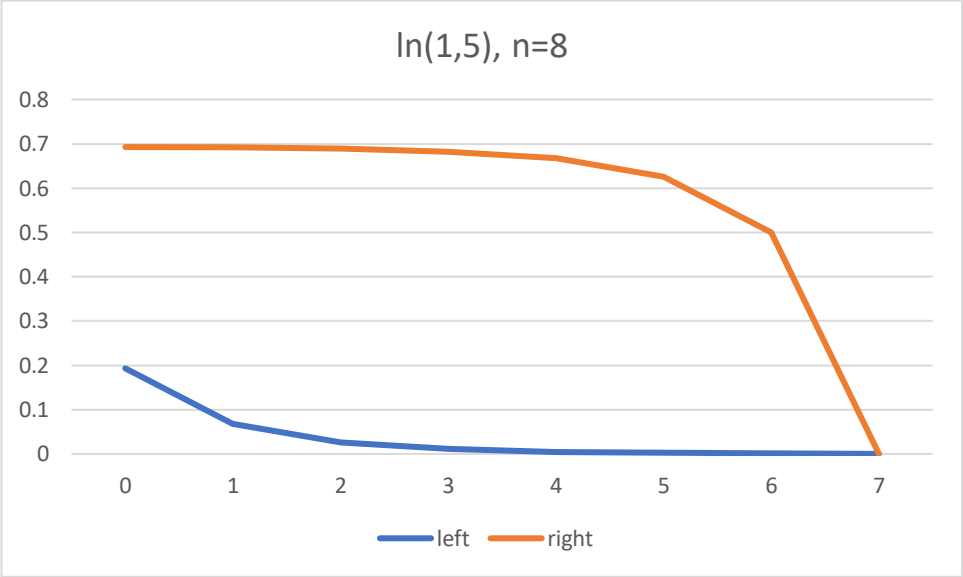

Рис. 8 погрешности натурального логарифма

На основе проведенных экспериментов можно сделать следующие наблюдения:

1. Для знакопеременных рядов (sin, cos) метод суммирования слева значительно превосходит метод суммирования справа. Левая сумма обеспечивает монотонное убывание погрешности и достигает точного значения уже на 6-7 членах ряда. Правая сумма при чтении снизу вверх показывает хаотичное поведение погрешности и требует 20 и более членов для достижения той же точности.

2. Для рядов с монотонно убывающими членами (ln(1+x)) оба метода показывают идентичные результаты. Это объясняется тем, что члены ряда убывают равномерно, и порядок сложения не влияет на потерю значимости.

3. Для рядов с растущими членами (exp(x)) оба метода показывают одинаковые результаты. Это связано с тем, что при больших значениях аргумента члены ряда сначала растут, а затем убывают, и оба метода накапливают сумму одинаково.

Заключение

В ходе выполнения лабораторной работы было разработано программное обеспечение для вычисления частичных сумм рядов Тейлора с использованием двух методов накопления: суммирования слева (sum_left) и суммирования справа (sum_right). Проведены эксперименты для функций. Можно сделать вывод, что для большинства практических задач, особенно при работе со знакопеременными рядами, рекомендуется использовать метод суммирования слева, так как он обеспечивает более быструю сходимость и меньшую погрешность при фиксированном количестве членов ряда. Метод суммирования справа может быть полезен только в специфических случаях, когда требуется минимизировать накопление ошибок округления при сложении чисел сильно различающихся порядков.

Литература

1) https://metanit.com/
2) Эффективный С. Профессиональное программирование – Роберт С. Сикорд
3) Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992

Приложение